

Docker — Theoretical Question Set

A student worksheet covering Docker fundamentals, Dockerfiles, images, networking, volumes, and Docker Compose.

Instructions: Part A is multiple choice — circle or write the letter of the best answer. Part B is short answer — 2 to 5 sentences per question is usually enough.

Part A — Multiple Choice

A1. Docker Fundamentals

1. What is the main difference between a Docker container and a virtual machine?
 - A. Containers include a full guest operating system, VMs do not
 - B. Containers share the host OS kernel, VMs virtualize hardware and run a separate OS
 - C. Containers are always slower to start than VMs
 - D. There is no meaningful difference
2. Which of the following best describes a Docker image?
 - A. A running instance of an application
 - B. A read-only template used to create containers
 - C. A virtual network between containers
 - D. A log file produced by a container
3. What happens to data written inside a container's writable layer when the container is removed?
 - A. It is automatically backed up by Docker
 - B. It is moved to the image
 - C. It is lost, unless it was stored in a volume or bind mount
 - D. It is copied to the host's home directory
4. Which command lists all running containers?
 - A. docker images
 - B. docker ps
 - C. docker container list --all-images
 - D. docker top

A2. Dockerfiles & Images

5. In a Dockerfile, what does the FROM instruction do?
 - A. Copies files from the host into the image
 - B. Specifies the base image to build on top of
 - C. Sets the working directory

D. Defines the container's entry command

6. Why is layer caching a good reason to COPY dependency manifests (e.g. requirements.txt, package.json) before copying the rest of the source code?

A. It's required by Docker and the build will fail otherwise

B. It has no effect on build speed

C. It lets Docker reuse the cached dependency-install layer when only app code changes

D. It reduces the number of containers created

7. What is the purpose of a multi-stage build?

A. To run multiple containers from one Dockerfile automatically

B. To build in one stage with all build tools, then copy only needed artifacts into a smaller final image

C. To connect two Dockerfiles together

D. To allow a Dockerfile to build on multiple operating systems at once

8. What is the key difference between CMD and ENTRYPOINT in a Dockerfile?

A. They are identical and interchangeable

B. CMD only works with Linux images, ENTRYPOINT only works with Windows images

C. CMD provides defaults that can be overridden at 'docker run', ENTRYPOINT sets the fixed main command (which CMD can supply arguments to)

D. ENTRYPOINT is deprecated in modern Docker

9. What does the EXPOSE instruction actually do?

A. It automatically publishes the port to the host

B. It documents which port the container listens on; it does not publish the port by itself

C. It opens a firewall rule on the host

D. It sets the container's IP address

A3. Networking & Volumes

10. By default, how can two containers on the same user-defined Docker network reach each other?

A. Only via the host's public IP address

B. By using each other's container/service name as a hostname

C. They cannot communicate without a load balancer

D. Only if both containers expose port 80

11. What is the main advantage of a named volume over a bind mount for persisting a database's data?

A. Volumes are managed by Docker and are portable across environments without relying on host filesystem paths

B. Bind mounts are always faster

C. Volumes cannot be backed up

D. There is no difference

12. Which flag maps port 8080 on the host to port 80 inside the container?

A. docker run -p 80:8080 image

B. docker run -p 8080:80 image

- C. `docker run --port 8080 image`
- D. `docker run -e PORT=8080 image`

A4. Docker Compose

13. What is the primary purpose of a `docker-compose.yml` file?

- A. To replace Dockerfiles entirely
- B. To define and run multi-container applications with a single configuration file
- C. To build a single container image faster
- D. To store container logs

14. In Docker Compose, what does `'depends_on'` guarantee by default (without a healthcheck condition)?

- A. That the dependency service is fully ready to accept connections
- B. Only that the dependency container has started, not that it's ready
- C. That the dependency will restart automatically forever
- D. Nothing — `'depends_on'` has no effect

15. What is the difference between `'docker compose down'` and `'docker compose down -v'`?

- A. There is no difference
- B. `'-v'` also removes the named volumes declared in the compose file, deleting persisted data
- C. `'-v'` runs the containers in verbose mode
- D. `'-v'` only stops containers without removing them

Part B — Short Answer & Discussion

Answer in your own words. These questions don't have a single 'correct' sentence — they're checking that you understand the concept well enough to explain it.

B1. Concepts

1. Explain, in your own words, the difference between an image and a container.
An image is a read-only template; a container is a running instance of that image.
2. What problem does containerization solve that virtual machines solve less efficiently?
Containers share the host kernel, so they use fewer resources and start faster than VMs.
3. Describe what 'It works on my machine' means as a problem, and how Docker addresses it.
Docker packages the app and its dependencies together, reducing environment differences.
4. What is a Docker registry (e.g. Docker Hub), and how does it relate to images?
A registry stores and distributes Docker images; Docker Hub is a public example.

B2. Building Images

5. Walk through what happens, step by step, when you run 'docker build .' — what does Docker do with each instruction in the Dockerfile?
Docker reads the Dockerfile and executes each instruction in order, creating image layers.
6. Why might a Dockerfile that installs dependencies with 'RUN' before copying application code build faster on repeated builds than one that copies everything first?
Docker can reuse the dependency layer when only the application code changes, making builds faster.
7. What is the difference between a JDK-based and a JRE-based Java image, and why would you use each?
JDK images are for development and compilation; JRE images are mainly for running Java apps.
8. Give one reason a team might prefer a small base image (e.g. 'alpine' or 'scratch') over a full OS image, and one potential downside of doing so.
Small images reduce size and attack surface, but they may contain fewer tools for debugging.

B3. Running & Managing Containers

9. What is the difference between stopping a container and removing a container?
Stopping keeps the container; removing deletes the container itself.
10. Describe a scenario where you would use 'docker exec' during development or debugging.
Use docker exec to enter a running container and inspect files or processes while debugging.
11. What information does 'docker logs' show you, and why is it useful when a container fails to start?
docker logs shows the main process output and helps find startup or application errors.

B4. Networking & Storage

12. Explain why containers on the default bridge network in older Docker versions couldn't resolve each other by name, but containers on a user-defined network can.
User-defined networks provide DNS name resolution between containers; the old default bridge did not.

13. Describe a real situation where you would need a volume rather than relying on the container's own filesystem.

A database can use a volume so its data survives container removal and recreation.

B5. Docker Compose

14. Explain how Docker Compose simplifies running an application made of multiple services (e.g. a web app and a database) compared to running each with separate 'docker run' commands.

Compose defines services and their settings in one file, so one command can run the whole app.

15. What is the purpose of environment variables in a docker-compose.yml file, and give an example of when you'd use one.

Environment variables provide configuration, such as REDIS_HOST=redis for the Redis service.